

Retrieval Augmented Generation (RAG) Systems Guide

Introduction

Large Language Models (LLMs) such as GPT, Gemini, Claude, Llama, Mistral, and Qwen are powerful reasoning and text generation systems. However, they have an important limitation: they only know information that was available during training and may generate incorrect or outdated answers.

Retrieval Augmented Generation (RAG) solves this problem by combining:

Large Language Model
+
Knowledge Retrieval

Instead of relying entirely on model memory, a RAG system retrieves relevant information from external knowledge sources before generating an answer.

This approach enables:

- More accurate responses
- Access to private knowledge bases
- Reduced hallucinations
- Up-to-date information
- Better explainability

Examples:

- LMS chatbots
- Enterprise knowledge assistants
- Customer support systems
- Internal company copilots
- Legal document assistants
- Medical information systems

Chapter 1: What Problem Does RAG Solve?

Consider a chatbot trained in 2024.

Question:

What courses are currently available in our LMS?

The model cannot know this information unless it was part of training data.

Without RAG:

```
graph TD;
    A[User Question] --> B[LLM];
    B --> C[Potential Hallucination];
```

With RAG:

```
graph TD;
    A[User Question] --> B[Retriever];
    B --> C[Relevant Documents];
    C --> D[LLM];
    D --> E[Accurate Answer];
```

The model answers using retrieved knowledge rather than guessing.

Chapter 2: High-Level RAG Architecture

A typical RAG pipeline consists of:

```
graph TD;
    A[Documents] --> B[Chunking];
    B --> C[Embeddings];
    C --> D[Vector Database];
    D --> E[Retriever];
    E --> F[ ];
```

LLM



Response

Each component has a specific responsibility.

Chapter 3: Documents and Knowledge Sources

Knowledge sources may include:

- PDF files
- Word documents
- CSV files
- Databases
- Websites
- Notion pages
- Confluence pages
- LMS course materials

Example LMS documents:

```
react_notes.pdf
nextjs_guide.pdf
machine_learning_basics.pdf
attendance_policy.pdf
internal_sop.pdf
```

The quality of a RAG system heavily depends on the quality of source documents.

Chapter 4: Document Loading

The first step is loading documents.

Popular document formats:

- PDF
- DOCX
- TXT
- HTML
- Markdown

Python example:

```
from langchain.document_loaders import PyPDFLoader

loader = PyPDFLoader("react_notes.pdf")
docs = loader.load()
```

Output:

```
Page 1
Page 2
Page 3
...
```

Chapter 5: Text Chunking

LLMs cannot process extremely large documents efficiently.

Documents are split into smaller pieces called chunks.

Example:

Original document:

```
5000 words
```

Chunked into:

```
Chunk 1
Chunk 2
Chunk 3
Chunk 4
...
```

Benefits:

- Faster retrieval
 - Better relevance
 - Reduced token usage
-

Chunk Size

Typical values:

```
300 tokens
500 tokens
800 tokens
1000 tokens
```

Popular strategy:

```
Chunk Size = 500
Chunk Overlap = 100
```

Chapter 6: Embeddings

Embeddings convert text into numerical vectors.

Example:

```
"React is a JavaScript library"
```

Becomes:

```
[0.23, -0.41, 0.91, ...]
```

These vectors capture semantic meaning.

Similar concepts have similar vectors.

Example:

```
React Tutorial
React Components
```

Their embeddings will be close in vector space.

Chapter 7: Embedding Models

Popular embedding models:

OpenAI

```
text-embedding-3-small  
text-embedding-3-large
```

Google

```
gemini-embedding
```

BAAI

```
bge-small  
bge-base  
bge-large
```

Sentence Transformers

```
all-MiniLM-L6-v2
```

For production systems, embedding quality is critical.

Chapter 8: Vector Databases

Embeddings must be stored somewhere.

Traditional SQL databases search exact values.

Vector databases search semantic similarity.

Popular options:

- Pinecone
- Weaviate
- Chroma
- Milvus

- Qdrant
 - pgvector
-

pgvector

PostgreSQL extension for vector search.

Example table:

```
CREATE TABLE chunks (  
  id UUID PRIMARY KEY,  
  content TEXT,  
  embedding VECTOR(768)  
);
```

Advantages:

- Works inside PostgreSQL
- Easy integration
- Open source

Excellent choice for LMS systems.

Chapter 9: Similarity Search

When a user asks a question:

```
How does useEffect work?
```

The question is converted into an embedding.

Retriever searches:

```
Vector Similarity
```

Nearest chunks are returned.

Example:

```
Chunk A -> Similarity 0.92
Chunk B -> Similarity 0.87
Chunk C -> Similarity 0.81
```

Top results are selected.

Chapter 10: Retrieval

Retriever finds relevant knowledge.

Workflow:

```
Question
  ↓
Embedding
  ↓
Vector Search
  ↓
Top K Chunks
```

Common values:

```
Top K = 3
Top K = 5
Top K = 10
```

Higher values provide more context but increase token usage.

Chapter 11: Prompt Construction

Retrieved chunks are inserted into prompts.

Example:

```
Context:
[Retrieved Chunks]

Question:
```


Explain React Hooks.

Answer:

This process grounds the model in actual knowledge.

Chapter 12: Generation

The LLM generates answers using:

Question
+
Retrieved Context

Example:

Question:
What is React?

Context:
React is an open-source JavaScript library...

Answer:
React is an open-source JavaScript library...

The answer becomes more accurate and less prone to hallucination.

Chapter 13: Metadata Filtering

Metadata improves retrieval quality.

Example metadata:

```
{  
  "course": "MERN",  
  "module": "React",  
  "role": "student"  
}
```

Query:

Show only MERN course content

Retriever applies filters before searching.

Chapter 14: Role-Based RAG

Role-based retrieval is critical for LMS systems.

Student:

Access:
- Notes
- Assignments
- Recorded Classes

Teacher:

Access:
- Notes
- Attendance Reports
- Evaluation Policies

Admin:

Access:
- Revenue Reports
- Internal SOPs
- Analytics

Architecture:

User
↓
Role Check
↓
Filtered Retrieval

↓
LLM

Chapter 15: Hybrid Search

Vector search is not always enough.

Hybrid search combines:

Semantic Search
+
Keyword Search

Benefits:

- Better accuracy
- Better recall
- Better ranking

Popular in enterprise systems.

Chapter 16: Re-ranking

Retriever may return imperfect results.

Re-rankers improve ordering.

Workflow:

Top 20 Results
↓
Re-ranker
↓
Best 5 Results

Popular models:

- BGE Reranker
 - Cohere Rerank
-

Chapter 17: LangChain

LangChain is a framework for LLM applications.

Features:

- Document loading
- Chunking
- Retrieval
- Agents
- Memory

Example:

```
from langchain.chains import RetrievalQA
```

LangChain simplifies RAG implementation.

Chapter 18: LangGraph

LangGraph enables agent workflows.

Example:

```
Question
↓
Retriever
↓
Planner
↓
Tool
↓
Answer
```

Benefits:

- Multi-step reasoning
 - State management
 - Agent orchestration
-

Chapter 19: AI Agents and RAG

Modern AI systems combine:

RAG
+
Tools
+
Memory
+
Reasoning

Agent capabilities:

- Search documents
- Query databases
- Call APIs
- Execute workflows

Example:

Student:
Show my pending assignments.

Agent:

Check Database
Retrieve Assignments
Generate Response

Chapter 20: RAG Evaluation

Important metrics:

Retrieval Accuracy

Did we retrieve the correct chunks?

Context Relevance

Was the context useful?

Answer Accuracy

Did the answer match the source?

Hallucination Rate

Did the model invent information?

Evaluation is essential in production systems.

Chapter 21: Common Challenges

Poor Chunking

Problem:

Context lost

Solution:

Smaller chunks
Proper overlap

Bad Embeddings

Problem:

Irrelevant retrieval

Solution:

Better embedding models

Large Context Costs

Problem:

High token usage

Solution:

Better retrieval
Re-ranking

Chapter 22: Building an LMS RAG Chatbot

Architecture:

```
graph TD;
    A[PDF Notes] --> B[Supabase Storage];
    B --> C[Ingestion Pipeline];
    C --> D[Chunking];
    D --> E[Embeddings];
    E --> F[pgvector];
    F --> G[Role-Based Retriever];
    G --> H[Gemini / GPT / Qwen];
    H --> I[Student Answer];
```

Features:

- Course Q&A
- Assignment support
- Attendance information
- Teacher resources
- Admin analytics

Chapter 23: Best Practices

1. Use high-quality documents.
2. Chunk intelligently.
3. Add metadata.

4. Implement role filtering.
 5. Use re-ranking.
 6. Monitor hallucinations.
 7. Evaluate retrieval quality.
 8. Version embeddings.
 9. Cache frequent queries.
 10. Secure sensitive documents.
-

Summary

Retrieval Augmented Generation (RAG) combines information retrieval with Large Language Models to create accurate, scalable, and knowledge-aware AI systems. By using chunking, embeddings, vector databases, retrieval, and role-based access control, organizations can build powerful assistants that answer questions using trusted knowledge sources rather than relying solely on model memory. RAG forms the foundation of modern enterprise AI systems, LMS chatbots, knowledge assistants, and agentic applications.